

# QuickSlot — Hiring Hackathon Problem Statement

**Role:** Flutter Mobile Developer (with backend skills) · **Format:** Live, individual · **Build time:** 6 hours, then demo + defense round

---

## Before the event (mandatory — do this at home)

Come with a fully working setup. No setup time will be given on the day:

- Flutter SDK installed, `flutter doctor` clean, Android emulator or a physical device working
- Runtime installed for your backend language of choice (Node / Python / Go / Java / Dart — anything)
- Git, your IDE, and any AI tools you plan to use — installed and logged in

## The problem

Build **QuickSlot** — a mini app for booking sports slots (badminton courts / turf grounds). Users browse venues, view time slots for a date, and book one.

**The one hard rule: a slot can never be double-booked.** If two users tap "Book" on the same slot at the same instant, exactly one succeeds and the other gets a clear in-app message. **We will test this live from two devices.**

## Part A — Backend (any language, any database)

A REST API with real persistence (SQLite / Postgres / MySQL / Mongo — your choice). Seed 3–5 venues, each with hourly slots from 6 AM to 10 PM.

| Endpoint                                   | What it does                                                                                                                    |
|--------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| GET /venues                                | List venues                                                                                                                     |
| GET /venues/{id}/slots?<br>date=YYYY-MM-DD | Slots for a date, with status                                                                                                   |
| POST /bookings                             | Book a slot for a user — must be concurrency-safe; return correct status codes for success / slot already taken / invalid input |
| GET /users/{id}/bookings                   | A user's bookings                                                                                                               |
| DELETE /bookings/{id}                      | Cancel a booking                                                                                                                |

Keep auth light: hardcoded users plus an `X-User-Id` header is acceptable. Do not burn time building full auth.

## Part B — Flutter app

1. **User select / login** — keep it simple
2. **Venue list** → **venue detail** — date picker + slot grid, available vs booked clearly visible
3. **Booking flow** — confirm → success; if the slot was just taken by someone else, show a graceful message and refresh the grid
4. **My Bookings** — list with cancel

Required everywhere: **loading, error, and empty states**. Any state management (Provider / Riverpod / Bloc / GetX) — you will be asked to justify your choice.

## Rules

1. **AI tools (Claude, ChatGPT, Cursor, Copilot...) are allowed and expected** — same as the real job. But you must understand every line. The defense round includes explaining randomly chosen parts of your code and making a small live change to it.
  2. Commit to git **at least every 45 minutes** with meaningful messages. One giant final commit = penalty.
  3. Official framework starters are fine; personal pre-built boilerplates are not.
  4. Everything must run locally on your laptop for the demo (backend + app together).
- 

## Bonus (attempt max 2, only after the core works end-to-end)

- Slot status updates via polling or websocket (a slot flips to "booked" on another phone without restarting)
- Offline read cache for My Bookings
- Unit tests for booking logic, or one widget test
- Dockerized backend
- Filter slots by time of day

## Deliverables

1. **Git repo** (monorepo is fine: /app, /server)
2. **README** containing: setup steps, a short architecture note (a paragraph + a photo of a whiteboard sketch is fine), what you cut and why, what you'd do with one more day, and an **AI usage note** — what you used AI for and one thing it got wrong that you caught and fixed
3. **10-min demo + ~15-min defense**

## How you'll be evaluated

- Working core booking flow, demoed on a device

- The live double-booking test (two judges, two phones, same slot)
- Backend & API quality: schema, validation, status codes, your concurrency approach
- Flutter code quality: structure, state management, no business logic inside widgets, error/empty/loading states handled
- Defense round: explain your decisions, walk through code we pick, implement one small new requirement live
- Judgment: commit history, README honesty, scope decisions

A smaller app that is correct, well-structured, and fully understood beats a feature-stuffed app you can't explain. Cutting scope deliberately — and saying so in the README — is a positive signal.

## **Sample day schedule**

9:00 kickoff + Q&A · 9:30–15:30 build (hard code freeze) · 15:45 onwards demos + defense in random order · results the same evening or next day